BÀI THỰC HÀNH – CRUD REST API cho eShop (Spring Boot + JPA + MariaDB) và kiểm thử bằng Postman

Thời lượng	120 phút
Hình thức	Cá nhân, làm trên máy phòng lab
Cơ sở dữ liệu	`eShop.sql` (dùng 2 bảng `category`, `product`)
Công cụ	JDK 17 · Spring Boot 3.2.12 · Gradle 8.2.1 (`C:\gradle-8.2.1`) · IntelliJ IDEA Ultimate · MariaDB · Postman
Nộp bài	1 file `MSSV_HoTen_RestApi.zip` (xem mục 7)

---

1. Mục tiêu

Sau buổi thực hành, sinh viên có thể:

1. Tạo project Spring Boot bằng Gradle, kết nối MariaDB qua Spring Data JPA.
2. Ánh xạ bảng có quan hệ khoá ngoại (`product` → `category`) thành entity `@ManyToOne`.
3. Xây dựng REST API đầy đủ CRUD theo đúng quy ước: **HTTP method**, **URL danh từ số nhiều**, **mã trạng thái** (200, 201, 204, 400, 404, 409).
4. Dùng **DTO** cho request/response, **Bean Validation** để kiểm tra dữ liệu, **`@RestControllerAdvice`** để trả lỗi dạng JSON thống nhất.
5. Viết Postman Collection có **biến môi trường**, **test script** tự kiểm tra kết quả, chạy toàn bộ bằng **Collection Runner**.

2. Kiến thức cần nhớ (đọc trong 5 phút)

REST dùng URL để chỉ **tài nguyên** (danh từ) và HTTP method để chỉ **hành động**:

Hành động	Method	URL	Body gửi lên	Mã thành công	Body trả về
Lấy danh sách	`GET`	`/api/products`	–	**200 OK**	mảng / trang dữ liệu
Lấy 1 bản ghi	`GET`	`/api/products/5`	–	**200 OK**	1 đối tượng
Thêm mới	`POST`	`/api/products`	JSON	**201 Created** + header `Location`	đối tượng vừa tạo
Cập nhật	`PUT`	`/api/products/5`	JSON	**200 OK**	đối tượng sau khi sửa
Xoá	`DELETE`	`/api/products/5`	–	**204 No Content**	rỗng

Mã lỗi	Khi nào
400 Bad Request	Dữ liệu gửi lên sai (thiếu tên, giá âm, JSON hỏng, id không phải số)
404 Not Found	Không tìm thấy bản ghi theo id
409 Conflict	Vi phạm ràng buộc nghiệp vụ (trùng tên danh mục, xoá danh mục đang có sản phẩm)

Các annotation chính: `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PathVariable`, `@RequestParam`, `@RequestBody`, `@Valid`, `ResponseEntity`, `@ResponseStatus`, `@RestControllerAdvice`, `@ExceptionHandler`.

---

3. Phân bổ thời gian

Thời gian	Nội dung	Mục
0:00 – 0:10	Tạo CSDL, tạo project, cấu hình kết nối	4.1 – 4.2
0:10 – 0:25	Entity, Repository	4.3
0:25 – 0:50	API Danh mục (Category)	4.4
0:50 – 1:20	API Sản phẩm (Product) – tìm kiếm, phân trang	4.5

Thời gian	Nội dung	Mục
1:20 – 1:35	Xử lý lỗi tập trung	4.6
1:35 – 1:55	Kiểm thử bằng Postman, chạy Collection Runner	5
1:55 – 2:00	Nén và nộp bài	7

Làm **theo thứ tự**. Mỗi khi xong một API, mở Postman gọi thử ngay, đừng để dồn đến cuối giờ.

---

4. Yêu cầu thực hiện

4.1. Chuẩn bị (0:00 – 0:05)

1. Mở IntelliJ (tab *Database*) hoặc HeidiSQL, chạy toàn bộ file **`eShop.sql`**. Kiểm tra: `category` có **15** dòng, `product` có **50** dòng.
2. Tạo project mới: *File → New → Project → Java*, **Build system: Gradle**, **Gradle DSL: Groovy**, **JDK 17**.
    - Name: `eshop-api` · GroupId: `com.example` · Gradle distribution: **Local installation →** **`C:\gradle-8.2.1`**
3. *Settings → Build Tools → Gradle*: kiểm tra **Gradle JVM = 17**. *Settings → Compiler → Annotation Processors*: tick **Enable annotation processing**.

4.2. Cấu hình (0:05 – 0:10)

`build.gradle` (cho sẵn – chép nguyên văn):

```groovy
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.2.12'
    id 'io.spring.dependency-management' version '1.1.6'
}

group = 'com.example'
version = '0.0.1-SNAPSHOT'

java {
    sourceCompatibility = JavaVersion.VERSION_17
}
```

````
configurations {
    compileOnly { extendsFrom annotationProcessor }
}

repositories { mavenCentral() }

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation
'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation
'org.springframework.boot:spring-boot-starter-validation'
    runtimeOnly 'org.mariadb.jdbc:mariadb-java-client'
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation
'org.springframework.boot:spring-boot-starter-test'
}

tasks.named('test') { useJUnitPlatform() }
```
````

`settings.gradle`: `rootProject.name = 'eshop-api'` → bấm **Load Gradle Changes**.

`src/main/resources/application.properties`:

````
```properties
server.port=8080
spring.datasource.url=jdbc:mariadb://localhost:3306/eshop
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
```
````

> ⚠️ Không viết chú thích `# ...` cùng dòng với giá trị trong file `.properties`. Đổi mật khẩu `root` theo máy của bạn.
````

Tạo lớp `com.example.eshopapi.EshopApiApplication` có `@SpringBootApplication` và hàm `main`. Chạy thử: console phải có dòng `Tomcat started on port 8080`, **không** có exception.

4.3. Entity và Repository (0:10 – 0:25)

Cấu trúc package bắt buộc:

```
None
com.example.eshopapi
├── entity        Category, Product
├── repository    CategoryRepository, ProductRepository
├── dto           CategoryRequest, CategoryResponse, ProductRequest,
ProductResponse, PageResponse, ErrorResponse
├── exception     NotFoundException, ConflictException, GlobalExceptionHandler
├── service       CategoryService, ProductService
└── controller    CategoryController, ProductController
```

Yêu cầu 1 – Entity. Viết 2 entity ánh xạ **đúng tên bảng, tên cột** trong `eShop.sql`:

Entity	Bảng	Thuộc tính → cột
`Category`	`category`	`id` (IDENTITY), `name`, `description`
`Product`	`product`	`id` (IDENTITY), `name`, `description`, `price` (`BigDecimal`), `quantity` (`Integer`), `imageUrl` → `image_url`, `category` → `category_id` (`@ManyToOne(fetch = LAZY)`), `createdAt` → `created_at` (`LocalDateTime`, **không cho cập nhật**)

Gợi ý cho `createdAt`: `@CreationTimestamp` + `@Column(name = "created_at", updatable = false)`.

Yêu cầu 2 – Repository.

- `CategoryRepository extends JpaRepository<Category, Long>` có thêm:
  - `boolean existsByNameIgnoreCase(String name)`
  - `boolean existsByNameIgnoreCaseAndIdNot(String name, Long id)` (dùng khi cập nhật)
- `ProductRepository extends JpaRepository<Product, Long>` có thêm:

- `boolean existsByCategoryId(Long categoryId)`
- `List<Product> findByCategoryId(Long categoryId)`
- Một phương thức `search(...)` viết bằng `@Query` (JPQL), lọc **đồng thời** theo: từ khoá trong tên (không phân biệt hoa thường), `categoryId`, `minPrice`, `maxPrice`; tham số nào là `null` thì **bỏ qua** điều kiện đó; nhận `Pageable`, trả `Page<Product>`.

Gợi ý mẫu điều kiện "null thì bỏ qua": `(:categoryId IS NULL OR p.category.id = :categoryId)`.

4.4. API Danh mục – `/api/categories` (0:25 – 0:50)

Yêu cầu 3 – DTO. Không trả entity trực tiếp ra JSON. Dùng Java `record`:

```java
public record CategoryRequest(
        @NotBlank(message = "Tên danh mục không được để trống")
        @Size(max = 100, message = "Tên tối đa 100 ký tự") String name,
        @Size(max = 255, message = "Mô tả tối đa 255 ký tự") String description)
{}

public record CategoryResponse(Long id, String name, String description) {}
```

Yêu cầu 4 – Đặc tả API:

#	Method & URL	Mô tả	Thành công	Lỗi phải xử lý
C1	`GET /api/categories`	Danh sách tất cả danh mục, sắp xếp theo `name` tăng dần	200	–
C2	`GET /api/categories/{id}`	Chi tiết 1 danh mục	200	404 nếu không có
C3	`POST /api/categories`	Thêm danh mục	**201** + header `Location: /api/categories/{idMới}`	400 thiếu tên · 409 trùng tên (không phân biệt hoa thường)
C4	`PUT /api/categories/{id}`	Sửa danh mục	200	404 · 400 · 409 trùng tên với danh mục **khác**
C5	`DELETE /api/categories/{id}`	Xoá danh mục	**204**	404 · **409 nếu danh mục đang có sản phẩm**
C6	`GET /api/categories/{id}/products`	Các sản phẩm thuộc danh mục	200	404 nếu danh mục không có

Ví dụ:

```http
POST /api/categories
Content-Type: application/json

{ "name": "Phụ kiện gaming", "description": "Tay cầm, ghế, bàn gaming"
}
```

```http
HTTP/1.1 201 Created
Location: /api/categories/16
Content-Type: application/json

{ "id": 16, "name": "Phụ kiện gaming", "description": "Tay cầm, ghế,
bàn gaming" }
```

Gợi ý trả 201 kèm `Location`:

```java
URI location = URI.create("/api/categories/" + created.id());
return ResponseEntity.created(location).body(created);
```

4.5. API Sản phẩm – `/api/products` (0:50 – 1:20)

Yêu cầu 5 – DTO:

```java
public record ProductRequest(
        @NotBlank(message = "Tên sản phẩm không được để trống") @Size(max = 150)
String name,
        @Size(max = 500) String description,
        @NotNull(message = "Giá không được để trống") @DecimalMin(value = "0",
message = "Giá phải >= 0") BigDecimal price,
        @NotNull(message = "Số lượng không được để trống") @Min(value = 0, message
= "Số lượng phải >= 0") Integer quantity,
        @Size(max = 500) String imageUrl,
        @NotNull(message = "Phải chọn danh mục") Long categoryId) {}

public record ProductResponse(Long id, String name, String description, BigDecimal
price, Integer quantity,
                              String imageUrl, Long categoryId, String
categoryName, LocalDateTime createdAt) {}
```

```
public record PageResponse<T>(List<T> content, int page, int size, long
totalElements, int totalPages) {}
```

Yêu cầu 6 – Đặc tả API:

#	Method & URL	Mô tả	Thành công	Lỗi phải xử lý
P1	`GET /api/products`	Tìm kiếm + phân trang + sắp xếp (bảng tham số bên dưới), trả `PageResponse<ProductResponse>`	200	400 nếu tham số sai kiểu (vd `page=abc`)
P2	`GET /api/products/{id}`	Chi tiết sản phẩm	200	404
P3	`POST /api/products`	Thêm sản phẩm	**201** + `Location`	400 dữ liệu sai · 404 nếu `categoryId` không tồn tại
P4	`PUT /api/products/{id}`	Sửa toàn bộ thông tin sản phẩm (**không** đổi `createdAt`)	200	404 sản phẩm / danh mục · 400
P5	`DELETE /api/products/{id}`	Xoá sản phẩm	**204**	404 · 409 nếu sản phẩm đã có trong `order_detail`

Tham số của P1 (tất cả không bắt buộc):

Tham số	Kiểu	Mặc định	Ví dụ
`keyword`	String	–	`samsung`
`categoryId`	Long	–	`3`
`minPrice`, `maxPrice`	BigDecimal	–	`1000000`, `5000000`
`page`	int	`0`	`1`
`size`	int	`10`	`5`
`sort`	String `thuộc_tính,asc|desc`	`id,asc`	`price,desc`

Ví dụ: `GET /api/products?keyword=samsung&sort=price,desc&size=3`

```
JSON
{
  "content": [
```

```
    { "id": 2, "name": "Samsung Galaxy S24 256GB", "description": "Exynos 2400,
...", "price": 22990000.00,
      "quantity": 20, "imageUrl": "https://placehold.co/400x300?text=Galaxy+S24",
      "categoryId": 1, "categoryName": "Điện thoại", "createdAt":
"2026-10-27T08:00:00" },
    { "id": 6,  "name": "Samsung Galaxy Tab S9 FE", "price": 9490000.00, "...":
"..." },
    { "id": 30, "name": "Samsung Odyssey G5 27 inch", "price": 6490000.00, "...":
"..." }
  ],
  "page": 0, "size": 3, "totalElements": 5, "totalPages": 2
}
```

> **Lưu ý:** `spring.jpa.open-in-view=false` nên phải đọc `product.getCategory().getName()` **bên trong** `@Transactional` (ở Service), hoặc dùng `JOIN FETCH` / `@EntityGraph`. Nếu chuyển entity sang DTO ở Controller, bạn sẽ gặp `LazyInitializationException`. Đây là chủ ý của bài.

4.6. Xử lý lỗi tập trung (1:20 – 1:35)

Yêu cầu 7. Mọi lỗi phải trả về JSON **cùng một định dạng**:

```java
public record ErrorResponse(LocalDateTime timestamp, int status, String error,
String message,
                            String path, Map<String, String> errors) {}
```

Ví dụ lỗi 400 khi validation:

```json
{
  "timestamp": "2026-10-27T09:15:02.123",
  "status": 400,
  "error": "Bad Request",
  "message": "Dữ liệu không hợp lệ",
  "path": "/api/products",
  "errors": { "name": "Tên sản phẩm không được để trống", "price": "Giá phải >= 0"
}
}
```

Ví dụ lỗi 404: `{ "status": 404, "error": "Not Found", "message": "Không tìm thấy sản phẩm id = 999", "errors": null, ... }`

Viết `GlobalExceptionHandler` (`@RestControllerAdvice`) xử lý **tối thiểu**:

Exception	Mã
`NotFoundException` (tự định nghĩa)	404
`ConflictException` (tự định nghĩa)	409
`MethodArgumentNotValidException` (lỗi `@Valid`)	400, có `errors` theo từng trường
`HttpMessageNotReadableException` (JSON hỏng, sai kiểu)	400
`MethodArgumentTypeMismatchException` (vd `/api/products/abc`)	400
`DataIntegrityViolationException` (vướng khoá ngoại/unique ở DB)	409

---

5. Kiểm thử bằng Postman (1:35 – 1:55)

5.1. Chuẩn bị

1. Tạo **Environment** tên `eshop-local`, biến `baseUrl` = `http://localhost:8080`. Chọn environment này ở góc phải.
2. Tạo **Collection** `MSSV_eShop_API` gồm 2 folder: `Categories`, `Products`.
3. Tất cả URL viết dạng `{{baseUrl}}/api/...`.
4. Request POST/PUT: tab *Body* → *raw* → *JSON*.

5.2. Biến dùng chung giữa các request

Request "tạo mới" phải **lưu id vừa tạo** vào biến collection để các request sau dùng lại. Viết ở tab **Scripts → Post-response** (Postman cũ: tab *Tests*):

```javascript
pm.test("Status 201 Created", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("Có id kiểu số", () => pm.expect(body.id).to.be.a("number"));
pm.test("Header Location đúng", () =>
    pm.expect(pm.response.headers.get("Location")).to.eql("/api/categories/" +
body.id));
pm.collectionVariables.set("newCategoryId", body.id);
```

Các request sau dùng `{{newCategoryId}}`, ví dụ `PUT {{baseUrl}}/api/categories/{{newCategoryId}}`.

5.3. Các test case bắt buộc

Mỗi request **phải có ít nhất 1 test kiểm tra status code**. Các dòng có dấu ★ phải thêm test kiểm tra **nội dung body**.

Folder `Categories` (sắp đúng thứ tự này):

#	Tên request	Request	Mong đợi
1	★ Lấy tất cả danh mục	`GET /api/categories`	200, mảng có **15** phần tử (lần chạy đầu)
2	★ Lấy danh mục id=3	`GET /api/categories/3`	200, `name` = `"Laptop"`
3	Danh mục không tồn tại	`GET /api/categories/999`	404
4	★ Thêm danh mục	`POST /api/categories {"name":"Phụ kiện gaming {{$timestamp}}","description":"Test"}`	201, lưu `newCategoryId`
5	★ Thêm thiếu tên	`POST /api/categories {"description":"x"}`	400, `errors.name` tồn tại
6	Thêm trùng tên	`POST /api/categories {"name":"laptop"}`	409
7	★ Sửa danh mục	`PUT /api/categories/{{newCategoryId}} {"name":"Gaming Gear {{$timestamp}}","description":"Đã sửa"}`	200, `description` = `"Đã sửa"`
8	Sửa trùng tên danh mục khác	`PUT /api/categories/{{newCategoryId}} {"name":"Laptop"}`	409
9	★ Sản phẩm của danh mục 5	`GET /api/categories/5/products`	200, có **4** sản phẩm, tất cả `categoryId` = 5

#	Tên request	Request	Mong đợi
10	Xoá danh mục đang có sản phẩm	`DELETE /api/categories/1`	409

Folder `Products`:

#	Tên request	Request	Mong đợi
11	★ Trang đầu	`GET /api/products`	200, `totalElements` = 50, `content` có 10 phần tử
12	★ Tìm theo từ khoá	`GET /api/products?keyword=samsung`	200, `totalElements` = 5, mọi `name` chứa "samsung" (không phân biệt hoa thường)
13	★ Lọc danh mục + khoảng giá + sắp xếp	`GET /api/products?categoryId=3&minPrice=13000000&sort=price,desc`	200, 3 sản phẩm, giá **giảm dần**
14	★ Phân trang	`GET /api/products?page=4&size=12`	200, `content` có **2** phần tử, `totalPages` = 5
15	Tham số sai kiểu	`GET /api/products?page=abc`	400
16	★ Thêm sản phẩm	`POST /api/products` (body bên dưới)	201, `categoryName` bắt đầu bằng `"Gaming Gear"` (tên đã sửa ở request 7), lưu `newProductId`
17	★ Thêm sai dữ liệu	`POST /api/products` `{"name":"","price":-5,"quantity":1,"categoryId":1}`	400, `errors` có `name` và `price`
18	Thêm với danh mục không tồn tại	`POST /api/products` `{"name":"X","price":1,"quantity":1,"categoryId":999}`	404

#	Tên request	Request	Mong đợi
19	★ Sửa sản phẩm	`PUT /api/products/{{newProductId}}` (đổi `price` thành `990000`)	200, `price` = 990000, `createdAt` **không đổi** so với lúc tạo
20	Id không phải số	`GET /api/products/abc`	400
21	Xoá sản phẩm	`DELETE /api/products/{{newProductId}}`	204
22	Xem lại sản phẩm đã xoá	`GET /api/products/{{newProductId}}`	404
23	Xoá danh mục vừa tạo	`DELETE /api/categories/{{newCategoryId}}`	204 (lúc này danh mục đã trống)
24	JSON hỏng	`POST /api/products` body: `{"name": "X",`	400

Body cho request 16:

```JSON
{
  "name": "Tay cầm Xbox Wireless",
  "description": "Kết nối Bluetooth, dùng cho PC và Xbox",
  "price": 1390000,
  "quantity": 15,
  "imageUrl": "https://placehold.co/400x300?text=Xbox+Controller",
  "categoryId": {{newCategoryId}}
}
```

Ví dụ test script cho request 13 (kiểm tra sắp xếp):

```JavaScript
pm.test("Status 200", () => pm.response.to.have.status(200));
const items = pm.response.json().content;
pm.test("Đúng 3 laptop giá >= 13 triệu", () =>
pm.expect(items).to.have.lengthOf(3));
pm.test("Giá giảm dần", () => {
    for (let i = 1; i < items.length; i++) {
        pm.expect(items[i - 1].price).to.be.at.least(items[i].price);
```

```
    }
});
```

Ví dụ test script cho request 19 (kiểm tra `createdAt` không đổi). Ở request 16, lưu thêm `pm.collectionVariables.set("newProductCreatedAt", body.createdAt);`, rồi ở request 19:

```javascript
pm.test("Status 200", () => pm.response.to.have.status(200));
const p = pm.response.json();
pm.test("Giá đã cập nhật", () => pm.expect(p.price).to.eql(990000));
// Cột DATETIME của MariaDB không lưu phần lẻ của giây → chỉ so sánh đến giây (19 ký tự đầu)
pm.test("createdAt giữ nguyên", () =>
    pm.expect(p.createdAt.substring(0, 19))
      .to.eql(pm.collectionVariables.get("newProductCreatedAt").substring(0, 19)));
```

5.4. Chạy toàn bộ

Bấm chuột phải vào collection → **Run collection** → *Run* (giữ đúng thứ tự). Chạy **2 lần liên tiếp**: cả 2 lần phải **xanh 100%**. Lần thứ hai giúp kiểm tra bộ test không phụ thuộc dữ liệu cũ; vì vậy tên danh mục có `{{$timestamp}}` và cuối collection có bước xoá dữ liệu test.

> Nếu test số 1 (đếm 15 danh mục) đỏ ở lần chạy thứ hai, nghĩa là request 23 chưa xoá được dữ liệu test của lần chạy trước. Hãy kiểm tra lại.

---

6. Yêu cầu nâng cao (cộng điểm, làm nếu còn thời gian)

#	Yêu cầu	Điểm cộng
N1	`PATCH /api/products/{id}/stock` body `{"delta": -3}`: tăng/giảm tồn kho; trả 409 nếu tồn kho sau khi trừ < 0	+0.5
N2	Chỉ cho phép `sort` theo `id`, `name`, `price`, `quantity`, `createdAt`; thuộc tính khác → 400	+0.5

#	Yêu cầu	Điểm cộng
N3	`GET /api/categories/stats`: mỗi danh mục kèm số sản phẩm và tổng giá trị tồn kho (`SUM(price*quantity)`), dùng JPQL `GROUP BY` trả về DTO	+1.0

---

7. Nộp bài

Nén thành **`MSSV_HoTen_RestApi.zip`**, gồm:

```
None
MSSV_HoTen_RestApi/
├── eshop-api/                          ← toàn bộ source (XOÁ thư mục build/,
.gradle/, .idea/ trước khi nén)
├── postman/
│   ├── MSSV_eShop_API.postman_collection.json     ← Export collection (định dạng
v2.1)
│   └── eshop-local.postman_environment.json       ← Export environment
└── screenshots/
    ├── runner-lan1.png                ← kết quả Collection Runner lần 1
    └── runner-lan2.png                ← kết quả Collection Runner lần 2
```

---

8. Tiêu chí chấm điểm (thang 10)

Tiêu chí	Điểm
Project chạy được, kết nối MariaDB, entity ánh xạ đúng (kể cả `@ManyToOne`, `created_at` không bị ghi đè)	1.5
Repository: query method + `search` bằng JPQL lọc đủ 4 điều kiện, có `Pageable`	1.0
API Category C1–C6 đúng method, URL, status code (201 + Location, 204)	2.0
API Product P1–P5, tìm kiếm/phân trang/sắp xếp đúng	2.0

Tiêu chí	Điểm
DTO + Bean Validation, không trả entity ra JSON	1.0
`GlobalExceptionHandler` trả lỗi JSON thống nhất đủ 6 loại exception	1.0
Postman: environment, biến collection, đủ 24 request có test script, Runner xanh 2 lần	1.5
Cộng yêu cầu nâng cao	tối đa +2.0 (điểm tổng không vượt 10)

Trừ điểm: trả entity trực tiếp (−0.5) · dùng `ddl-auto=update/create` làm thay đổi bảng (−0.5) · URL dạng động từ như `/api/getProducts`, `/api/deleteProduct` (−1.0) · nộp kèm thư mục `build/` (−0.25).

---

9. Lỗi thường gặp

Hiện tượng	Cách xử lý
`Unsupported class file major version 65`	Gradle JVM đang là 21 → chọn **17**
`Access denied for user 'root'`	Sai mật khẩu trong `application.properties`
`Table 'eshop.product' doesn't exist`	Chưa chạy `eShop.sql` hoặc URL sai tên DB
`LazyInitializationException: could not initialize proxy ... no Session`	Chuyển entity → DTO bên trong phương thức `@Transactional` ở Service, hoặc `JOIN FETCH`
POST trả **415 Unsupported Media Type**	Postman chưa chọn *Body* → *raw* → ***JSON***
POST trả 400 dù dữ liệu đúng, `name` nhận `null`	Thiếu `@RequestBody` trước tham số
`@Valid` không có tác dụng	Thiếu dependency `spring-boot-starter-validation` hoặc thiếu `@Valid`
JSON trả về bị lặp vô hạn / `ByteBuddyInterceptor`	Đang trả entity trực tiếp → dùng DTO

Hiện tượng	Cách xử lý
`Name for argument of type [java.lang.Long] not specified`	Chạy bằng IntelliJ build (không phải Gradle) nên thiếu cờ `-parameters` → *Settings* → *Gradle* → *Build and run using:* ***Gradle***, hoặc ghi rõ `@PathVariable("id")`
Tiếng Việt trong JSON bị lỗi font	Kiểm tra DB tạo với `utf8mb4` (đã có trong `eShop.sql`)